

```
"""
```

```
=====
GENERADOR DE REPORTES BCRA – MoliPay / Money Life S.R.L.
=====
```

Régimen Informativo PSCCP – genera automáticamente los tres archivos `xlsx` que deben subirse al SISCEN del BCRA:

1. `apartado_a.xlsx` – Saldos y cuentas por rango
2. `apartado_b.xlsx` – Operativa mensual (transferencias, canales)
3. `padron.xlsx` – Nómina de clientes personas jurídicas

Basado en la estructura de los archivos de muestra provistos.

Requiere: `openpyxl` (`pip install openpyxl`)

USO BÁSICO

```
from bcra_generator import generar_apartado_a, generar_apartado_b, generar_pa

generar_apartado_a(datos_a, "apartado_a.xlsx")
generar_apartado_b(datos_b, "apartado_b.xlsx")
generar_padron(clientes_pj, "padron.xlsx")
```

Cada función recibe los datos ya agregados desde la base de MoliPay (ver bloque "MAPEO DESDE LA BASE" al final del archivo).

```
=====
"""
```

```
from openpyxl import Workbook
from typing import Iterable, Mapping
```

```
# =====
# APARTADO A – Saldos por rango de legajo
# =====
# Estructura por fila:
#   LEGAJOS (7 dígitos) | MONTO | MONTO_SIN_3_CEROS | COL_AUX
#
# Bloques de legajo detectados en la muestra (31 filas cada uno):
#   1000001..1000031 -> monto en $ + cantidad de cuentas (col C)
#   2000001..2000031 -> monto en $ + identificador técnico en col D
#   3000001..3000031 -> reservado (ceros)
#   4000001..4000031 -> reservado (ceros)
#   5000001..5000031 -> reservado (ceros)
#   6000001..6000031 -> reservado (ceros)
```

```

#
# IMPORTANTE: todos los valores se escriben como STRING (BCRA los lee así).

def generar_apartado_a(datos: Mapping[str, dict], ruta_salida: str) -> None:
    """
    datos: dict con clave = legajo (str, 7 dígitos) y valor = dict con:
        {
            "monto":      int|str,    # monto en pesos
            "cantidad":   int|str,    # cantidad de cuentas / valor col C (opcion
            "aux":        str,        # valor columna D (opcional, para bloque 2X
        }
    Los legajos no provistos se completan con cero.
    """
    wb = Workbook()
    ws = wb.active
    ws.title = "Sheet1"

    # Cabecera
    ws.append(["LEGAJO", "MONTO", "MONTO EN CUENTA SIN 3 CEROS ULTIMOS", None])

    # Genera los 6 bloques × 31 filas = 186 filas
    for bloque in range(1, 7):
        for i in range(1, 32):
            legajo = f"{bloque}{i:06d}"                # 1000001, 1000002, ...
            registro = datos.get(legajo, {})
            monto = str(registro.get("monto", 0))
            cantidad = registro.get("cantidad", "")
            aux = registro.get("aux", "")

            # Bloque 1: monto + cantidad en col C, col D vacía
            # Bloque 2: monto + col C vacía, aux (identificador) en col D
            # Bloques 3-6: reservados (ceros o vacíos)
            if bloque == 1:
                ws.append([legajo, monto, str(cantidad) if cantidad != "" else ""
            elif bloque == 2:
                ws.append([legajo, monto, "", str(aux) if aux else ""])
            else:
                ws.append([legajo, "0", "0", ""])

        wb.save(ruta_salida)

# =====
# APARTADO B – Operativa mensual
# =====
# Sin cabecera. Códigos fijos:
#

```

```

# 5001000 -> transferencias recibidas (cantidad, monto)
# 5002000 -> transferencias enviadas (cantidad, monto)
# 5003000 -> pagos recibidos (cantidad, monto)
# 5004000 -> pagos enviados (cantidad, monto)
# 2020220 -> fila especial con 5 campos: (código, 8, 00009, 0, 0)
# 6011020..6042010 -> operativa por canal (solo cantidad, monto vacío)
#
# NOTA: los códigos 6XXXXXXX suelen ser pares "canal+resultado" (aprobado/rechaza

```

```

CODIGOS_B_MONTO = [
    "5001000", "5002000", "5003000", "5004000",
]

```

```

CODIGOS_B_SOLO_CANTIDAD = [
    "6011020", "6012010", "6013020", "6014010",
    "6021020", "6022010", "6023020", "6024010",
    "6031020", "6032010",
    "6041020", "6042010",
]

```

```

def generar_apartado_b(datos: Mapping[str, dict], ruta_salida: str) -> None:
    """
    datos: dict con clave = código y valor = dict:
        {"cantidad": int, "monto": int} # monto solo para los 5XXXXXXX
    Los códigos ausentes se completan en cero.
    """
    wb = Workbook()
    ws = wb.active
    ws.title = "Sheet1"

    # Bloque 5XXXXXXX - cantidad + monto
    for cod in CODIGOS_B_MONTO:
        r = datos.get(cod, {})
        cantidad = str(r.get("cantidad", 0))
        monto = str(r.get("monto", 0))
        ws.append([cod, None, None, cantidad, monto])

    # Fila especial 2020220
    r = datos.get("2020220", {})
    ws.append([
        "2020220",
        str(r.get("col_b", 8)),
        str(r.get("col_c", "00009")),
        str(r.get("cantidad", 0)),
        str(r.get("monto", 0)),
    ])

```

```

# Bloque 6XXXXXXX – solo cantidad
for cod in CODIGOS_B_SOLO_CANTIDAD:
    r = datos.get(cod, {})
    cantidad = str(r.get("cantidad", 0))
    ws.append([cod, None, None, cantidad, None])

wb.save(ruta_salida)

# =====
# PADRÓN – Personas jurídicas
# =====
# Sin cabecera. Estructura fija:
#
# Col A: tipo de documento    (siempre "11" = CUIT)
# Col B: CUIT sin guiones     (11 dígitos)
# Col C: sufijo                (siempre "00")
# Col D: vacía
# Col E: razón social          (rellenada con espacios a 80 caracteres)
# Col F: flag                  ("0")
# Col G: código postal         (formato argentino, ej "C1426BBS")
# Col H: tipo de persona       ("10", "20", ...)

def generar_padron(clientes: Iterable[dict], ruta_salida: str) -> None:
    """
    clientes: iterable de dicts:
        {
            "cuit":          str,    # 11 dígitos sin guiones
            "razon_social":  str,    # nombre legal
            "cp":            str,    # código postal argentino
            "tipo_persona":  str,    # "10" (SRL), "20" (SA), etc.
        }
    """
    wb = Workbook()
    ws = wb.active
    ws.title = "Sheet1"

    for c in clientes:
        razon_padded = c["razon_social"].ljust(80)[:80]    # ancho fijo 80
        ws.append([
            "11",
            str(c["cuit"]),
            "00",
            None,
            razon_padded,
            "0",

```

```

        c["cp"],
        str(c["tipo_persona"]),
    ])

    wb.save(ruta_salida)

# =====
# MAPEO DESDE LA BASE DE MoliPay
# =====
# Estas son las queries lógicas que el programador debe implementar contra la
# base de MoliPay para llenar cada estructura. Ajustar a los nombres reales
# de tablas/columnas del sistema.
#
# --- APARTADO A ---
# Legajos 1000001..1000031 = saldos totales por rango de saldo:
#     SELECT rango_id, SUM(saldo), COUNT(*)
#     FROM cuentas_pago
#     WHERE fecha_corte = :ultimo_dia_mes
#     GROUP BY rango_id
#
# Legajos 2000001..2000031 = mismo agrupado con identificador técnico (col D)
#
# --- APARTADO B ---
# 5001000 = transferencias RECIBIDAS del mes (COUNT, SUM)
# 5002000 = transferencias ENVIADAS del mes (COUNT, SUM)
# 5003000 = pagos RECIBIDOS del mes (COUNT, SUM)
# 5004000 = pagos ENVIADOS del mes (COUNT, SUM)
# 6XXXXXXX = cantidades por canal (web, app, API, etc.) aprobadas/rechazadas
#
# --- PADRÓN ---
#     SELECT cuit, razon_social, codigo_postal, tipo_societario
#     FROM clientes
#     WHERE tipo_persona = 'juridica'
#     AND estado = 'activo'
#
# =====
# EJEMPLO DE USO (test)
# =====

if __name__ == "__main__":
    # Ejemplo Apartado A
    datos_a = {
        "1000001": {"monto": 494230, "cantidad": 534},
        "1000002": {"monto": 490103, "cantidad": 536},
        "2000001": {"monto": 494230, "aux": "432000100000000000000000"},
    }

```

```
generar_apartado_a(datos_a, "apartado_a.xlsx")

# Ejemplo Apartado B
datos_b = {
    "5001000": {"cantidad": 7270, "monto": 1132252},
    "5002000": {"cantidad": 10303, "monto": 717246},
    "6011020": {"cantidad": 9},
}
generar_apartado_b(datos_b, "apartado_b.xlsx")

# Ejemplo Padrón
clientes = [
    {"cuit": "30718679873", "razon_social": "GADIEL SRL",
     "cp": "S2200GQG", "tipo_persona": "10"},
    {"cuit": "30683054646", "razon_social": "CLINICA NOGUERA SA",
     "cp": "C1426BBS", "tipo_persona": "20"},
]
generar_padron(clientes, "padron.xlsx")

print("Archivos generados: apartado_a.xlsx, apartado_b.xlsx, padron.xlsx")
```